

INSTITUCIÓN UNIVERSITARIA DE COLOMBIA

SESIÓN 1 — REFUERZO PYTHON Y ÁLGEBRA LINEAL

I. DATOS GENERALES DE LA ASIGNATURA

Asignatura:	Inteligencia Artificial II
Período Académico:	V Semestre
Créditos:	3 créditos
Duración en Horas:	40 horas
Docente:	Amaury Giovanni Méndez Aguirre

INTRODUCCIÓN: LA CEGUERA COMPUTACIONAL Y EL ÁLGEBRA

Bienvenidos al curso. Como estudiantes de quinto semestre, ya no podemos ver las imágenes simplemente como "fotos". Para una máquina, una imagen no tiene semántica; es estrictamente un espacio matemático continuo que ha sido discretizado (muestreado) en una cuadrícula.

El procesamiento de imágenes y la Visión por Computadora (Computer Vision) son, en su esencia más pura, **Álgebra Lineal Aplicada**. Las redes neuronales convolucionales (CNNs) que estudiaremos más adelante no hacen más que aplicar transformaciones lineales (multiplicaciones de matrices y productos punto) seguidas de funciones no lineales. Hoy sentaremos las bases matemáticas y algorítmicas de cómo se representan y manipulan estas estructuras usando tensores en Python.

1. Representación: De Matrices a Tensores

Una imagen en escala de grises es una **Matriz (A)** de dimensiones $M \times N$, donde M es el número de filas (altura) y N el número de columnas (anchura). Cada elemento $A_{i,j}$ (píxel) representa la intensidad lumínica, comúnmente en un rango discreto de 8 bits: $[0, 255]$.

Una imagen a color (RGB) trasciende la matriz bidimensional y se convierte en un **Tensor de orden 3**, con dimensiones $M \times N \times C$ (donde C representa los 3 canales de color: Rojo, Verde y Azul).

Representación Visual (5x5)

0	255	255	255	0
255	0	0	0	255
255	0	128	0	255
255	0	0	0	255
0	255	255	255	0

≡

Matriz A (5x5)

0	255	255	255	0
255	0	0	0	255
255	0	128	0	255
255	0	0	0	255
0	255	255	255	0

TALLER ANALÍTICO 1: INDEXACIÓN Y TENSORES (15 MIN)

Respondan las siguientes preguntas teóricas discutiendo en grupo:

1. En la Matriz A de arriba, ¿cuál es el valor exacto del elemento $A_{2,3}$ (Asumiendo índice base 0 para i y j)? Justifique qué representa visualmente.
2. Si tenemos un Tensor RGB de dimensiones $1080 \times 1920 \times 3$, ¿cuántos valores individuales (bytes) se están almacenando en memoria para esa única imagen sin comprimir? Realice el cálculo.

Espacio para cálculos analíticos...

2. Operaciones Lineales: Brillo y Contraste

Modificar el brillo y contraste de una imagen no es más que aplicar una transformación lineal escalar a la matriz original. La ecuación fundamental es:

$$A_{nueva} = \alpha * A + \beta$$

- **Escalar α (Ganancia / Contraste):** Una multiplicación escalar deforma el rango (histograma). Si $\alpha > 1$, aumenta el contraste; si $0 < \alpha < 1$, disminuye.
- **Escalar β (Sesgo / Brillo):** Sumar una constante desplaza todos los valores de la matriz, subiendo o bajando la luminosidad general.

Riesgo Matemático (Clipping): Al hacer estas operaciones, los valores pueden superar 255 o caer por debajo de 0. En álgebra estricta esto es válido, pero en imágenes de 8-bits causará desbordamiento (overflow). Debemos usar funciones de "saturación" (clipping) para acotar los resultados al intervalo [0, 255].

```
import numpy as np

# Matriz original simulando una imagen 3x3 muy oscura
A = np.array([
    [10, 20, 10],
    [15, 30, 15],
    [10, 20, 10]
], dtype=np.float32)

# Parámetros lineales
alpha = 2.5 # Aumento de contraste (Multiplicación escalar)
beta = 50.0 # Aumento de brillo (Suma escalar a todos los elementos)

# Transformación lineal: Anueva = a*A + b
A_nueva = (alpha * A) + beta

# Acotamiento (Clipping) para mantener formato de imagen de 8 bits
A_nueva = np.clip(A_nueva, 0, 255).astype(np.uint8)

print("Matriz Resultante:")
print(A_nueva)
```

TALLER DE LABORATORIO 1: TRANSFORMACIONES AFINES (20 MIN)

Reto en Python: Imaginen que trabajan en software para radiografías. Las imágenes están sobreexpuestas (casi completamente blancas).

1. Abran su IDE de Python. Creen una matriz de prueba 5x5 usando `np.random.randint(200, 255, (5,5))`.
2. Apliquen matemáticamente una reducción de contraste del 50% y disminuyan el brillo en 50 unidades.
3. Asegúrense de usar `np.clip()` y convertir a enteros `np.uint8`.
4. Impriman la matriz original y la procesada para comparar.

3. Transformaciones Espaciales y Transposición

A nivel espacial, podemos alterar la geometría de la imagen. La operación más elemental es la **Matriz Transpuesta (A^T)**.

La transposición intercambia filas por columnas ($A^T_{i,j} = A_{j,i}$). En el procesamiento de imágenes, transponer una matriz equivale a reflejar la imagen sobre su diagonal principal (una rotación combinada con un espejo).

Otra operación vital en Inteligencia Artificial es el **Aplanamiento (Flattening)**. Las Redes Neuronales Densas (Perceptrón Multicapa) no aceptan matrices 2D; requieren vectores columna (1D). El proceso de "Flatten" toma una matriz $M \times N$ y la convierte en un vector de tamaño $(M*N) \times 1$.

Matriz A

1	2	3
4	5	6

→ Transposición (A^T) →

Matriz A^T

1	4
2	5
3	6

```
import numpy as np

A = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

# Transpuesta usando NumPy
A_transpuesta = A.T

# Aplanamiento (Vectorización para Redes Neuronales)
vector_1D = A.flatten()

print("Forma original:", A.shape)           # Salida: (2, 3)
print("Forma transpuesta:", A_transpuesta.shape) # Salida: (3, 2)
print("Vector plano:", vector_1D)           # Salida: [10 20 30 40 50 60]
```

TALLER ANALÍTICO 2: TRANSFORMACIONES (15 MIN)

1. Si transponemos una matriz identidad I de 4×4 , ¿cuál es la matriz resultante? Explique geoméricamente por qué.
2. Una imagen RGB tiene una forma de $(200, 200, 3)$. Si la "aplanamos" para introducirla en la capa de entrada de una Red Neuronal, ¿cuántas neuronas exactas deberá tener esa capa de entrada para recibir el vector?

Espacio para justificación...

4. El Producto Hadamard vs Producto Punto (Kernels)

El álgebra lineal nos ofrece diferentes formas de multiplicar matrices, y es vital no confundirlas al programar:

- **Producto Hadamard (Element-wise):** Se multiplica elemento por elemento ($C_{i,j} = A_{i,j} * B_{i,j}$). Se usa comúnmente en IA para aplicar **Máscaras Binarias** (recortar siluetas). En Python se hace con el operador `*` estándar.
- **Multiplicación Matricial (Dot Product):** Combina filas y columnas ($C = A \cdot B$). En Python se hace con `np.dot()` o el operador `@`.

Introducción a la Convolución: Las Redes Neuronales Convolucionales (CNN) usan matrices pequeñas (usualmente 3x3) llamadas **Kernels o Filtros**. Estos Kernels se "deslizan" por la imagen, realizando una suma de productos elementales en cada vecindario local para detectar bordes o texturas.

Sección de Imagen (I)

100	100	100
100	200	100
100	100	100

*

Kernel (K) de Realce

0	-1	0
-1	5	-1
0	-1	0

```
import numpy as np

# 1. Producto Hadamard (Aplicación de Máscara)
imagen = np.array([[200, 50], [255, 10]])
mascara_binaria = np.array([[1, 0], [1, 0]])
# Mantiene columna izq, borra der

resultado_mascara = imagen * mascara_binaria # Elemento a elemento
print("Resultado Máscara:\n", resultado_mascara)

# 2. Multiplicación Matricial Clásica
resultado_dot = np.dot(imagen, mascara_binaria)
```

TALLER DE LABORATORIO FINAL: PROGRAMANDO UN KERNEL (25 MIN)

Misión Práctica: Construyan un simulador básico de convolución para una matriz de 3x3 (sin usar librerías avanzadas como OpenCV, solo matemáticas en Python/NumPy).

1. Creen en NumPy la matriz "Sección de Imagen (I)" y el "Kernel (K)" mostrados en el diagrama superior.
2. Escriban el código para calcular el valor central resultante si superponemos el Kernel sobre la Imagen. *(Pista: Deben multiplicar la matriz 'I' por la matriz 'K' usando el producto Hadamard y luego sumar TODOS los valores de la matriz resultante usando `np.sum()`).*
3. Impriman el valor del píxel central calculado. Este es el principio fundamental de cómo un filtro detecta texturas en Deep Learning.